

Project Report

ClassLLM - Authentication Feature

Created By Claude-GPT-Mini: Idan Harel, Rom Waidergoren, Ilay Amsalem, Shon Grinberg



This document covers our first feature work in the project ClassLLM : Authentication and Authorization feature.

1 Overview

ClassLLM is a project developed during the "Software Engineering with LLM" course as part of our Computer Science degree. In this project we managed to create an online chatbot that will be accessible to both students and teachers and would be able to help students learn some topics of their courses, sharpen their understanding and test their knowledge without the need of human staff or additional lecturing hours. Our team's first task in this project was to create the authentication service that allows users to login the system as teacher or student and authorize actions based this role. Another task we had to create a topics generator service that can label each markdown material document containing parts of the course materials with the relevant topics this content related to and generate a visual topics tree that represents the topics hierarchy.

2 Place in the Workflow

After launching the application in order to get access to the restricted pages (related to the user's role) there is a need for signing in. This stage is the authentication stage and after that the user can proceed to all the other services in the system.

3 Feature Requirements

The feature should have 4 functionalities:

- **Signup and Onboarding:** This functionality should let new user create an account using Google SSO and choose courses and his role (student or teacher).
- **Login Authentication:** User that already created account should be able to login to his user by the Google SSO interface.
- **Role Based Authorization:** Based on the user's role, he will get an access to specific pages that defined as accessible for this role and be restricted from other roles' pages.
- **API interface:** This functionality could be used by other services via API in order to set other features authorization and restrict those features from unwanted/unsigned users.

4 Implementation Details

In order to satisfy the authentication, onboarding, and role-based access requirements, we implemented the following functions and components:

1. **signInWithGoogle():** This function initiates user authentication using Google OAuth through Firebase Authentication. It attempts to sign in using a popup-based flow and automatically falls back to a redirect-based flow if popups are blocked. Upon successful authentication, a Firebase user session is created and an ID token becomes available for subsequent client-side and server-side operations.
2. **AuthProviderClient:** This component centralizes authentication state on the client. It listens for authentication state changes, loads the corresponding user profile from Firestore, and determines whether onboarding is required. It exposes the authenticated Firebase user, the loaded profile, and helper actions such as sign-in, sign-out, and profile refresh to the rest of the application.
3. **AuthRedirector:** This component is mounted at the root level of the application and performs automatic redirection once authentication state is resolved. Authenticated users are redirected away from neutral routes (such as the login page) to either the onboarding page or their role-specific dashboard, preventing unnecessary navigation steps.
4. **RoleGuardClient:** This component enforces role-based access control on the client. It ensures that unauthenticated users are redirected to the login page, users with incomplete profiles are redirected to onboarding, and users attempting to access routes outside their assigned role are redirected to the appropriate dashboard.
5. **verify_id_token(idToken):** This server-side function verifies Firebase ID tokens using the Firebase Admin SDK. It validates token authenticity and extracts the associated user identifier (UID). Requests containing invalid, expired, or missing tokens are rejected, ensuring that only authenticated users can access protected backend or micro-service endpoints.
6. **get_user_role(uid):** This function resolves the role of an authenticated user. In the web application, the role is read from the user's Firestore profile. In backend micro-services, the role may be retrieved from Firebase custom claims associated with the user. Only the roles `student` and `teacher` are accepted as valid.

7. **Test Token Authentication Route():** To support automated testing, a test-only authentication mechanism was implemented. A dedicated server route can mint Firebase custom tokens and optionally create user profiles when enabled via environment configuration. This allows end-to-end tests to exercise authenticated flows without relying on external OAuth providers. This mechanism is disabled by default and intended strictly for non-production environments.

5 Work Stages

After receiving the task we tried at first using the firebase LLM studio to generate the system entirely. This method didn't went well as the LLM didn't understand all the project requirements and add some unwanted feature (such login with Email etc.). We tried debugging it but it started adding a lot of unnecessary code so we updated our initial prompt and PRD and started all over again from scratch. After this generation we got a lot accurate version was closer to what we wanted but still required a lot of debug and fixes. We realized that firebase does not debug as good as we wanted so we moved to the Copilot which make a lot better work. this fixes took us most of this stage working time, some of it done using the Copilot and some manually. Our code was merged and we presented it in class, and got two more requirements from our feature:

- **Interactive via the API:** The feature should be interacted using an API.
- **Finding user role without calling the firestore database:** For fast and simple authorization method we needed to use the UID to authorize the user's role.

After that we got another requirement in order to understand the task better we arranged a meeting with Michael and verified our understanding and what we should do. Michael explained the task to us and we asked some questions that helped us to fully understood and go implementing it. We various LLM models to implement this and also to document all the specifications needed.

6 Results and Conclusions

In the end we got a working full authentication and authorization service that handles the sign-in and the roles restrictions that can be interacted with each outside service for use.

Main conclusion:

- **Different models are good at different kind of tasks:** As we learned in class, each model was trained on different data leading to different performance on some kind of tasks. In the project we first used Firebase that is based on Gemini but we found out that for our specific task of debugging the copilot performed better. For other tasks (such as writing the PRD or the specifications) the Gemini performed better than the others. So in each kind of task we used the correct tool for it, and along the usage we got more experience in this model choosing.

7 Reflection

Throughout the work on this project we learned a lot, had the chance to improve our software engineering skills, practice LLM usage and working in team. It was not easy for us especially because there are some tools we used in this project for the first time, but we think that this challenge was for the better.

In the end we succeeded doing two tasks instead of one and performed two demos - that made us very proud. In order to make the course better we suggest to create a task explaining page that will specify the requirement for each task to avoid confusion. In addition we suggest starting the integrations between different teams at an earlier stage of the course.

Another thing we think that will improve the course is implementing the use of Claude code in the course (the recent updates look very impressive).

At the bottom line we are very glad we took this course - we think that we learned important topics that will help us succeed in the software engineering with LLM world.